

Progn + More Functions

CS-3180

Global Namespace

- Everything we have done so far
- Existed in the global namespace
- Access data/functions globally
- `defparameter`, `defun` etc
- Let's build a program with better scoping

Paradigms

- Even in functional languages
- A degree of imperativeness is sometimes necessary
- We saw this before
- If we just execute a Lisp file, it evaluates things sequentially

Progn

- If you've learned one thing, its that everything is a function
- Sequential evaluation has also been functionalized
- `progn` function
- Sequentially evaluates all of its arguments
- Returns the last evaluation

Progn

- `(read-line)` get's returned
- Let's play around with that

```
(progn
  (format t "Hello world~%")
  (format t "My name is Bob~%")
  (format t "What is your name?~%")
  (finish-output)
  (read-line))
```

Side Effects

- ProgN is almost exclusively used for side effects
- MUST opt-in if we want that behavior
- Throw away any intermediate returns
- Hopefully, they shouldn't have any, like a print
- Pure Functions with no side-effects are the default

Scoping

- Your entire program is basically wrapped in `progn`
- Note that `defparameter` will still define globals
- Haven't really "scoped" our values
- Fix this later, but it's a start

```
1 (progn
2   (format t "What is your name?~%")
3   (finish-output)
4   (defparameter *name* (read-line)))
5
6 (format t "Name: ~S~%" *name*)
```

Implicit Progn

- Many functions in Lisp give you an IMPLICIT progn
- defun is one of these functions
- if is NOT
- Let's see both of those
- main is generally impure, let's write one

Main

- `defun` will still create global function
- We'll just call `main`

```
1 (defun square (value)
2     (format t "squaring value...~%")
3     (* value value))
4
5 (defun main ()
6     (format t "Hello world~%")
7     (format t "4 squared is ~S~%" (square 4)))
8
9
10 (main)
11
```

When

- Kind of like an if, but re-framed
- When 1st argument is `non-nil`, evaluate the 2nd
- Evaluate to `nil` otherwise
- Implicit ProgN

```
1 (defun foo (x)
2   (when (> x 0)
3     (print "Positive")
4     (* x 2)))
5
6 (format t "Value was: ~S~%" (foo 5))
7 (format t "Value was: ~S~%" (foo -5))
```

Unless

- When 1st argument IS `nil`, evaluate the 2nd
- Opposite of `when`
- Implicit `ProgN`
- Equivalent to `(when (not test) body)`

```
1 (defun my-sqrt (value)
2   (unless (< value 0)
3     (print "square rooting number...~%")
4     (sqrt value)))
5
6 (format t "Value was: ~S~%" (my-sqrt 5))
7 (format t "Value was: ~S~%" (my-sqrt -5))
```

Cond

- If-else-if-else block
- This will always work
- May not always want the jackhammer for the nail
- Lazily evaluates all of its `test` forms
- Each FULL FORM is one argument to `cond`, with a `progn`
- Let's see this

Cond

- Catch-all becomes `t`
- Each arg is of the form `(test body)`

```
1 (defun get-letter-grade (score)
2   (cond
3     ((>= score 90) "A")
4     ((>= score 80) "B")
5     ((>= score 70) "C")
6     (t "F")))
7
8 (defun main ()
9   (format t "Your letter grade was ~S~%"
10    (get-letter-grade 94))
11   (format t "Your letter grade was ~S~%"
12    (get-letter-grade 35)))
13
14 (main)
```

Cond ProgN

- ProgN here can complicate things
- Formatting gets difficult

```
1
2 (defun get-letter-grade (score)
3   (cond
4     ((>= score 90) (format t "Excellent~%" "A"))
5     ((>= score 80) (format t "Good Job~%" "B"))
6     ((>= score 70) (format t "...~%" "C"))
7     (t (format t ":(~%" "F")))
8
9 (defun main ()
10   (format t "Your letter grade was ~S~%"
11     (get-letter-grade 94))
12   (format t "Your letter grade was ~S~%"
13     (get-letter-grade 35)))
14
15 (main)
```

Let

- Scoped Variables*, finally
- Constant Name Bindings
- Body of the let function -> becomes the scope of the variable
- Implicit **progn**
- Returns the last value as a result
- Start with some small examples

Let

- Pay close attention to the ()s
- One of these ()s feels redundant

```
1
2 (defun get-letter-grade (score)
3   (cond
4     ((>= score 90) (format t "Excellent~%" "A")
5     ((>= score 80) (format t "Good Job~%" "B")
6     ((>= score 70) (format t "...~%" "C")
7     (t (format t ":(~%" "F"))))
8
9 (defun main ()
10   (let ((grade (get-letter-grade 97)))
11     (format t "Hi Bob~%")
12     (format t "You got the following grade: ~S~%" grade))
13
14 (main)
```

- You can actually define multiple variables
- Parallel Binding -> The coolest feature of Lisp
- Data must be mutually exclusive
- **name** doesn't rely on **grade**
- Use **let*** if you need that behavior

```
1 (defun get-letter-grade (score)
2   (cond
3     ((>= score 90) (format t "Excellent~%" "A")
4     ((>= score 80) (format t "Good Job~%" "B")
5     ((>= score 70) (format t "...~%" "C")
6     (t (format t ":(~%" "F"))))
7
8 (defun main ()
9   (let ((grade (get-letter-grade 97))
10        (name "Bob")))
11     (format t "Hi ~S~%" name)
12     (format t "You got the following grade: ~S~%" grade))
13
14 (main)
```

SetQ

- Evil imperative keyword
- Update a local binding
- Combine with **when** for a guard clause

```
1 (defun abs-square (value)
2   (when (< value 0)
3     (setq value (abs value))))
4   (* value value))
5
6 (format t "Value was: ~S~%" (abs-square -5))
7 (format t "Value was: ~S~%" (abs-square -3))
8 (format t "Value was: ~S~%" (abs-square 2
```

Summary

- Lisp is starting to get difficult
- Need to practice this, it get's easier
- Be mindful when you use an implicit `progn`
- All the functions we've discussed use it in some way

Questions?